

Lab Reflection — WalkMates

Lab: 2

Pair: Adam and Carl

Repo: [GitHub Repo](#)

1. What we did

The first step was to run `mvn clean test jacoco:report` to generate a report of the code coverage. This allowed us to record the line and branch coverage for `PricingCalculator`. Once the report was generated, we were able to identify areas of the code that were not being tested and focus our efforts on writing tests for those areas. Our focus was on ensuring that all branches were covered.

The next step in the process was to optimize the tests using **Mutation testing**. We ran a pitest and saw that all mutants on `PricingCalculator` were killed, which led us to the next step of isolating components with mocking. Our two main targets were `BookingService` and `SeekerService`. We wrote three tests for `SeekerService` covering **success**, **failure**, and **timeout** and one test for `BookingService` verifying the confirmation notification sent on a successful booking.

The last thing we did was a **Regression selection** for the **weekend-surcharge** with a focus on identifying those tests that were affected by the new feature. We created the prioritization based on which tests had a connection to the pricing calculation and gave those tests a higher priority, and tests not connected lower priority.

2. What we found

Activity 3.3

A test could "cover" the surcharge line yet miss the bug if it does not test for a booking of exactly 480 minutes. A booking that is more than 480 minutes would still meet the criteria of the if-statement and thus "cover" the surcharge line.

Activity 4.1

When we ran `mvn clean test org.pitest:pitest-maven:mutationCoverage`, we found that all mutants on `PricingCalculator` were killed.

Generally, we found that many mutants may survive even though coverage is high.

3. AI use (be honest — it doesn't lower your grade)

AI was used in this assignment to get a better understanding of what we were supposed to do and to explain different concepts. We were unsure how to perform the mutation testing, and AI gave us a better understanding of how to set up the mocks, what inject does, and what we could expect from running the tests we wrote. Generative AI was also used to finalize some tests where we had already written the majority of the test.

4. Judgment

The AI tools did not decide which tests to prioritize regarding the weekend-surcharge feature. This is something we wanted to stay in control of ourselves.

5. What we'd test next

Future work could be to continue with mutation testing for **BookingService** and **SeekerService**. Our focus for this assignment was to kill all mutants on **PricingCalculator** which we did, but there are still mutants that are not killed for other services.
